

Forest Fire Early Warning Sensor Network: A Discrete-Event Simulation Study Using OMNeT++, INET, and FLoRa

Alp Kaan Özgül

Student ID: 2638096

Department of Computer Engineering
Middle East Technical University
Northern Cyprus Campus
Email: alp.ozgul@metu.edu.tr

Yasin Bora Buğdacı

Student ID: 2584753

Department of Computer Engineering
Middle East Technical University
Northern Cyprus Campus
Email: bora.bugdaci@metu.edu.tr

Abstract—This interim project proposal report presents the design, modeling, and implementation plan for a Forest Fire Early Warning Sensor Network simulation project. The system deploys smoke, heat, humidity, and wind-speed sensors across a forested region, communicating through LoRa/LoRaWAN gateways to a central monitoring station. The simulation will be developed as a discrete-event model using OMNeT++, with INET providing protocol and networking support and FLoRa enabling realistic LoRa/LoRaWAN communication modeling. The report details how each compulsory course topic will be meaningfully used into the forest fire detection scenario. Stochastic inputs are modelled using the probability distributions covered in the course, including the Poisson, exponential, normal, Bernoulli, binomial, geometric, and uniform distributions, with variate generation based on the inverse transform method, convolution method, and empirical distribution sampling. This project aims to evaluate the responsiveness and dependability of the early warning network under varying environmental stress, sensor density, and traffic conditions.

1 INTRODUCTION AND SELECTED PROJECT DESCRIPTION

Forest fires are severe threat to ecosystems, human beings, and economic assets. Therefore, early detection of fire events is critical to minimising damage, yet remote forested areas are often inaccessible for continuous human activities. Internet of Things (IoT) sensor networks offer a good alternative: autonomous, low-power devices can continuously monitor environmental parameters and transmit warnings over long-range, low-power wireless links.

This project proposes in which smoke detectors, temperature sensors, humidity sensors, and wind-speed sensors are distributed across a forest area. These sensor nodes communicate via LoRa/LoRaWAN technology through one or more gateways to a central fire monitoring server. Under normal conditions, nodes transmit periodic telemetry reports at low duty cycles. When sensor readings exceed predefined fire-risk thresholds, alarm messages are generated and send to the monitoring system.

The primary motivation for simulating this system is to evaluate the *timeliness and reliability* of alarm delivery under

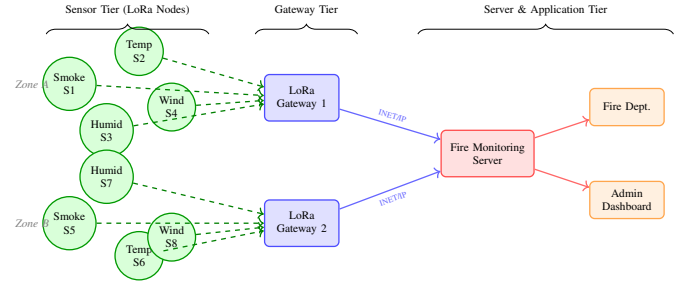


Fig. 1. System architecture of the Forest Fire Early Warning Sensor Network showing the three-tier IoT topology: sensor nodes communicate via LoRa to gateways, which forward data over IP (INET) to the central fire monitoring server. The server dispatches alerts to the fire department and administrative dashboards.

realistic conditions, including dense alarm traffic, wireless channel contention, gateway congestion, and varying network configurations.

The simulation will be developed using OMNeT++ as the discrete-event simulation engine, INET for IP-based backend communication between gateways and the monitoring server, and FLoRa for modelling LoRa physical-layer and LoRaWAN MAC-layer behaviour. All stochastic inputs are modelled using the probability distributions and variate generation techniques covered in the CNG-476 course lectures.

1.1. System Architecture and Technical Illustration

The proposed system follows a three-tier IoT architecture consisting of a *sensor tier*, a *gateway tier*, and a *server tier*. Fig. 1 illustrates the overall system topology and communication flow.

Sensor Tier. Multiple sensor nodes monitor smoke, temperature, humidity, and wind. When fire-risk limits are exceeded, nodes transmit alarm signals using high-priority LoRaWAN transmission.

Gateway Tier. Gateways receive, demodulate, and forward sensor data to the backend via IP networks. Queue management and service capacity are critical for alarm situations.

Server Tier. A central server processes and classifies data to validate fire conditions and dispatch notifications. Communication between gateways and the server uses the INET.

1.2. System Model and Simulation Objectives

1.2.1 System Entities and Resources: The simulation model includes the following principal entities:

- **Sensor Nodes** (N nodes, configurable): generate periodic telemetry and event-triggered alarm messages based on stochastic environmental models.
- **LoRa Channel:** models wireless propagation, path loss, interference, and collision behaviour using the LoRa physical-layer parameters defined in FLoRa.
- **LoRa Gateways** (G gateways, configurable): receive, demodulate, queue, and forward sensor packets.
- **Gateway Queue:** a finite-capacity service queue at each gateway, modelled as a single-server queue with FIFO discipline.
- **Backend IP Network:** INET-based connection between gateways and the monitoring server.
- **Fire Monitoring Server:** the sink entity that processes incoming data and evaluates alarm conditions.

1.2.2 Key Events: The simulation is driven by the following discrete events. Telemetry generation which stands for periodic sampling and transmission by each sensor node, Alarm generation for limit-triggered transmission when sensor inputs indicates fire risk, Packet arrival at gateway, represents arrival of a LoRa frame at the gateway receiver, service start/completion for beginning and end of packet and Collision event for data arrival at the monitoring server. Collision events and Packet delivery to server are other key events.

1.2.3 Simulation Objectives: The primary objectives of the simulation study are:

- 1) Evaluate the *end-to-end alarm delivery delay* from sensor detection to server reception under varying fire-event intensities.
- 2) Analyse the *packet delivery ratio* for both routine telemetry and urgent alarm traffic.
- 3) Study the *gateway queue conditions* (average queue length, waiting time, utilisation) under normal and conditions with high stress.
- 4) Assess the impact of *sensor density*, *number of gateways*, and *spreading factor allocation* on network reliability.
- 5) Compare alternative *reporting policies* (fixed-interval vs. adaptive vs. event-driven) in terms of detection speed and energy efficiency.
- 6) Quantify *collision probability* and its effect on alarm reliability during simultaneous multi-sensor fire detection.

1.3. Detailed Plan for Implementing Course Topics

1.3.1 Simulation Methodology and Modelling: The forest fire early warning system will be modelled at a *message-*

level abstraction, where the fundamental simulation entities are sensor nodes, gateways, the wireless channel, and the monitoring server. The real-world system is represented as a network of communicating modules in OMNeT++.

This project follows the standard simulation process: (i) problem definition, (ii) conceptual model design, (iii) computational model design, translation into an OMNeT++ network description (NED files) and C++ simple modules, (iv) verification to model, (v) validation by comparing queue metrics against M/M/1 analytical predictions, and (vi) experimentation and output analysis.

Assumptions. Sensors are stationary in a 2D flat area using log-distance path loss. Gateways support single-channel reception per spreading factor, while the backend IP link uses a fixed delay.

1.3.2 Random Number Generation: In our project, Uniform(0,1) random numbers will be used as the foundation for generating all other random variates through the inverse transform method and convolution method (covered in the Variates lecture). These numbers will be used to the sensor telemetry intervals and fire occurrences, environmental readings, package transmissions success and Bernoulli-based alarm and transmission decisions. Additionally, they facilitate discrete uniform selections for collision back-off slots. To ensure statistical independence and reproducibility, each component is assigned a distinct RNG stream, as shown in the lectures.

1.3.3 Monte Carlo Simulation: To obtain statistically meaningful results, the simulation will be executed as a Monte Carlo experiment consisting of repeated independent runs. Specifically:

- Each parameter configuration will be run with at least **30 independent replications**, each using a different initial seed for the pseudo-random number generator, to ensure sufficient sample size for confidence interval estimation.
- Across replications, the seeds for all RNG streams will be varied systematically using OMNeT++'s seed management functionalities (*seed-set* configuration).
- Multiple **parameter scenarios** will be explored, varying:
 - Number of sensor nodes: $N \in \{20, 40, 60, 80, 100\}$.
 - Number of gateways: $G \in \{1, 2, 3\}$.
 - Fire event rate: $\lambda_{\text{fire}} \in \{0.001, 0.005, 0.01, 0.02\}$ events per second.
 - Reporting policy: fixed-interval, adaptive, and event-driven.
- All results from replications will be used to compute sample means, standard deviations and etc.

1.3.4 Probability Models: All probability distributions used in this project are taken from the CNG-476 probability and variates lectures. Table I summarises the all distribution methods and their roles in the simulation.

1.3.5 Inverse Transformation or Equivalent Sampling: As covered in the Variates lecture, the **inverse transform method** is the primary technique for generating non-uniform random variates from Uniform(0,1) inputs. This method is applied in our project as follows:

TABLE I
PROBABILITY DISTRIBUTIONS USED IN THE PROJECT

Distribution	Role in the Simulation
Poisson (λ)	Number of fire ignition events in a fixed time interval; number of packet arrivals at the gateway per unit time.
Exponential (λ)	Inter-arrival times of fire events (as the continuous counterpart of the Poisson process); inter-transmission times of periodic sensor telemetry; service times at the gateway queue.
Normal (μ, σ^2)	Ambient temperature readings and humidity readings under stable forest conditions. Additive sensor measurement noise.
Bernoulli (p)	Whether a single sensor correctly detects a fire event (detection probability p_d); whether a transmitted packet is corrupted by channel error (bit error probability p_e); whether a sensor produces a false alarm in a given sampling cycle.
Binomial (n, p)	Number of sensors (out of n in an affected zone) that successfully detect a fire event; number of successful packet deliveries out of n transmission attempts.
Geometric (p)	Number of transmission attempts needed until first successful delivery of a packet (modelling retransmissions).
Uniform (a, b)	Random back-off interval after a collision (continuous uniform); initial sensor placement jitter within deployment zones; random alarm transmission delay (jitter) to spread bursty alarm packets.
Discrete Uniform	Selection of a random back-off slot from $\{1, 2, \dots, W\}$ after collision; random selection of spreading factor from a candidate set during initial assignment.
Empirical	If real-world sensor data or environmental datasets are available, a piecewise-linear empirical distribution will be fitted to the observed values and used as an alternative input model for sensor readings (e.g., wind speed data that does not fit a standard parametric distribution).

Exponential variate generation. Fire event inter-arrival times and telemetry intervals $X \sim \text{Exp}(\lambda)$ are generated using the inverse CDF method taught in class:

$$X = -\frac{1}{\lambda} \ln(1 - U), \quad U \sim \text{Uniform}(0, 1). \quad (1)$$

Since $1 - U$ is also $\text{Uniform}(0, 1)$, this simplifies to $X = -\frac{1}{\lambda} \ln(U)$, noting that the LCG never generates exactly 0.0, so the logarithm is safe.

Continuous uniform variate generation. Back-off intervals $Y \sim \text{Uniform}(a, b)$ are generated as:

$$Y = a + (b - a) \cdot U, \quad U \sim \text{Uniform}(0, 1). \quad (2)$$

Discrete distribution variates via inverse transform. For the Bernoulli, discrete uniform, and geometric distributions, the inverse transform method for discrete distributions is applied, as taught in the lecture. Given a $\text{Uniform}(0, 1)$ value U , the variate X is determined by finding the smallest x such that

$F(x) \geq U$, where F is the CDF. For example, a $\text{Bernoulli}(p)$ variate is:

$$X = \begin{cases} 1 & \text{if } U \leq p, \\ 0 & \text{if } U > p. \end{cases} \quad (3)$$

A geometric(p) variate (number of trials until first success) is generated as:

$$X = \left\lceil \frac{\ln(1 - U)}{\ln(1 - p)} \right\rceil. \quad (4)$$

Convolution method. As covered in the lecture, the binomial(n, p) variate is generated by summing n independent Bernoulli(p) variates:

$$X = \sum_{i=1}^n Y_i, \quad Y_i \sim \text{Bernoulli}(p). \quad (5)$$

Similarly, the Poisson(λ) variate is generated using the convolution-based algorithm from the lecture: generate exponential inter-arrival times $E_i = -\frac{1}{\lambda} \ln(U_i)$ and count the number of arrivals until the cumulative sum exceeds 1, i.e., find N such that $\prod_{i=1}^{N+1} U_i < e^{-\lambda} \leq \prod_{i=1}^N U_i$.

Normal variate generation. As discussed in the “Other Techniques” section of the Variates lecture, normal variates $Z \sim \mathcal{N}(0, 1)$ are generated from pairs of $\text{Uniform}(0, 1)$ numbers using:

$$Z_0 = \sqrt{-2 \ln U_1} \cos(2\pi U_2), \quad Z_1 = \sqrt{-2 \ln U_1} \sin(2\pi U_2), \quad (6)$$

where $U_1, U_2 \sim \text{Uniform}(0, 1)$. A general $\mathcal{N}(\mu, \sigma^2)$ variate is then obtained as $X = \mu + \sigma Z$.

These variate generation routines will be implemented as reusable C++ utility functions within the OMNeT++ project to demonstrate explicit understanding of the course material, rather than relying solely on built-in library functions.

1.3.6 Stochastic Processes: Fire event arrivals are modeled as a Poisson process with a rate λ_{fire} , while telemetry packets from sensor nodes follow exponentially distributed inter-transmission times. Fire detection at each sensor is represented by Bernoulli trials with probability p_d , creating dense alarm traffic. Gateway packet processing is modeled as an exponential random variable with rate μ , facilitating M/M/1 queuing analysis. Finally, the number of transmission attempts required for successful delivery follows a geometric distribution.

1.3.7 Queueing Analysis: Each LoRa gateway is modelled as a single-server queueing system:

Queueing model. Packets arriving at a gateway enter a FIFO queue. The gateway forwards one packet at a time. Under the assumption that packet arrivals follow a Poisson process and service times are exponentially distributed, this corresponds to an **M/M/1/K** queue (finite buffer of capacity K).

Analytical baseline. Before running the full simulation, we will compute the theoretical M/M/1 steady-state metrics for a simplified single-gateway scenario.

Key quantities to measure in the simulation are Average number of packets in the queue (L_q), average waiting time in

the queue (W_q), server utilisation (ρ), packet loss probability. Comparison of queue behaviour under normal telemetry load vs. dense alarm load.

1.3.8 Discrete-Event Simulation: This project is suitable for DES modelling because its state changes are instantaneous and event driven. Which means system states change only when specific events occur.

State variables maintained by the simulation include: each sensor node's operational mode (idle, sensing, transmitting, alarm), the gateway queue status, the wireless channel status (idle, busy, collision), and the server's processed alarm count.

1.3.9 Event Scheduling:

- **TelemetryGenerationEvent:** scheduled at each sensor node. The inter-generation time is drawn from $\text{Exp}(1/\bar{T})$ where $\bar{T}$ is the mean reporting period. Generated via the inverse transform: $t = -\bar{T} \cdot \ln(U)$.
- **FireIgnitionEvent:** scheduled globally. Inter-arrival times are drawn from $\text{Exp}(\lambda_{\text{fire}})$ following the Poisson process model. Upon occurrence.
- **AlarmGenerationEvent:** scheduled at individual sensor nodes when the current reading exceeds the risk limit.
- **ServiceStartEvent:** triggered when a packet reaches the head of the gateway queue and the server is idle.
- **CollisionBackoffEvent:** after a collision, each colliding node waits for a random time drawn from $\text{Uniform}(0, T_{\text{max}})$ or selects a random slot from $\text{DiscreteUniform}(1, W)$.
- **TimeoutEvent:** scheduled at the sender after transmission; if no approval arrives before expiry, a retransmission is triggered again.

1.3.10 Time-Stepped versus Trace-Driven Perspective:

Time-stepped relevance. Environmental state variables (ambient temperature, humidity) could be updated using a time-stepped approach at fixed intervals (e.g., every simulated minute).

Trace-driven relevance. If publicly available real-world environmental datasets (e.g., historical forest temperature and humidity records from weather stations) are obtained, they can be used as trace-driven inputs. In this case, the empirical distribution approach from the course lectures might be applied.

1.3.11 Wireless Network Modelling Fundamentals: The wireless communication model incorporates the following elements:

- **LoRa physical layer:** modelled using FLoRa's implementation of LoRa chirp spread-spectrum modulation. Key parameters include spreading factor (SF7–SF12), bandwidth (125 kHz), coding rate, and transmission power.
- **Collision model:** two transmissions on the same SF and channel that overlap in time are considered collided. FLoRa's collision resolution logic determines the result.
- **Gateway reception:** gateways can receive on multiple SFs simultaneously (one demodulator per SF), modelling realistic multi-channel gateway hardware.

- **Communication flow:** uplink-dominated traffic from sensor nodes to gateways, with optional downlink approval from the gateway to the node.

1.3.12 OMNeT++ Implementation: Using OMNeT++ as the core engine, we define the project through NED files to integrate sensor nodes, gateways, and a central server. Key modules include a sensor application for data generation, a gateway with custom exponential queuing for bottleneck analysis, and a global fire generator for Poisson-process ignitions. The omnetpp.ini file manages the design approach by automating parameter sweeps and replications to support rigorous Monte Carlo experimentation.

1.3.13 INET Integration: INET provides the backend infrastructure by modeling the IP-based connection between gateways and the server. It handles host configurations, routing protocols, and standard networking functions to ensure realistic data transport and propagation delays. This allows the simulation to accurately bridge LoRaWAN wireless traffic with the central server for final data processing.

1.3.14 FLoRa Integration: Flora is used to model the LoRaWAN physical and MAC layers it captures behaviors like collisions, interference, and the capture effect. In our project, Flora enables accurate simulation of sensor-to-gateway communication, supporting multiple gates and ensuring that high-priority alarm traffic is evaluated for reliability and delays during high density events.

1.3.15 Input Analysis: To make sure the simulation is valid, stochastic inputs will be looked at using distribution fitting, and chi-squared tests will be used to see how well the fit works. Histograms and QQ-plots will give you visual diagnostics by comparing real data to theoretical PDFs and quantiles. If parametric models don't work, a piecewise-linear empirical distribution will be made, and the inverse transform method will be used to create variates. Finally, a sensitivity analysis will check how strong the results are by trying out different distributions or real-world inputs.

1.3.16 Output Analysis: Simulation results are analyzed using at least 30 independent replications to ensure statistical validity. We construct 90/100 confidence intervals and discard initial warm-up data to focus on steady-state behavior. Performance across scenarios is visualized using box plots and error bars, while paired t-tests determine the significance of observed differences. Final data processing is handled via OMNeT++ analysis tools and Python to generate comprehensive statistical metrics.

1.4. Proposed Performance Metrics

The following performance metrics will be collected and analysed:

- 1) **End-to-End Alarm Delay (D_{alarm}):** the time elapsed from fire event occurrence to alarm reception at the monitoring server. This is the most critical metric for an early warning system.
- 2) **Packet Delivery Ratio (PDR):** the fraction of transmitted packets (both telemetry and alarm) successfully

received at the server, reflecting overall network reliability.

- 3) **Alarm Detection Rate:** the fraction of fire events for which at least one alarm packet is successfully delivered within a specified time window.
- 4) **Average Gateway Queue Length (L_q):** the mean number of packets waiting in the queue.
- 5) **Average Gateway Waiting Time (W_q):** the mean time a packet spends in the queue before service.
- 6) **Gateway Utilisation (ρ):** the fraction of time the gateway server is busy with processing other packets.
- 7) **Collision Rate:** the fraction of transmitted packets that have collision in the LoRa channel.
- 8) **False Alarm Rate:** the rate of alarm messages generated by sensor rather than real fire events, controlled by the Bernoulli false-alarm probability.
- 9) **Throughput:** the total number of successfully delivered packets per unit time.

1.5. Implementation Plan

There are four main phase in the implementation:

Phase 1: Environment Setup and Baseline Network.

Install and configure OMNeT++ 6.x, INET 4.x, and FLoRa. Build a baseline LoRaWAN network with N sensor nodes, G gateways, and a single server using FLoRa's existing modules. Verify basic packet transmission and reception. Implement the C++ variate generation utility functions.

Phase 2: Application Logic Development. Develop the custom sensor application module implementing: This phase involves creating a sensor module that generates regular updates and realistic environmental data while using threshold detection to trigger alarms. It includes multiple reporting modes—fixed, adaptive, and event-driven—to manage data flow. Additionally, a fire generator will simulate random ignitions that influence sensor readings within a specific area.

Phase 3: Queueing and Monitoring Infrastructure. Extend the gateway module with a custom queue (FIFO, finite capacity K) that logs ordering, waiting time, and drop statistics. Implement exponential service times for M/M/1 analysis and validation. Add INET-based backend communication between gateways and the server.

Phase 4: Experimentation result and their analysis.

Configure Monte Carlo experiments in `omnetpp.ini` with parameter sweeps over N , G , λ_{fire} , spreading factor allocation, and reporting policy. Execute all replications, collect results, and perform statistical analysis and perform input and output analysis.

Three primary comparison scenarios are planned:

- 1) **Reporting policy comparison:** fixed-interval telemetry vs. adaptive (increased frequency when readings approach threshold) vs. pure event-driven (alarm only).
- 2) **Network density comparison:** varying the number of sensor nodes ($N = 20$ to 100) with fixed gateway count to study scalability and congestion.

- 3) **Gateway deployment comparison:** single gateway vs. multi-gateway configurations to assess the benefit of area's redundancy on alarm performance.

1.6. Expected Challenges and Initial Solutions

Challenge 1: Realistic fire propagation modelling. Realistically modelling fire is one challenge. *Initial solution:* Fire events are simplified as point events with a configurable influence radius that changes sensor values.

Challenge 2: Gateway congestion during alarm bursts. When a fire event triggers alarms simultaneously from different sensors, the gateway queue may overflow. *Initial solution:* implement priority queueing at the gateway.

Challenge 3: Simulation warm-up and steady-state identification. Determining when the system reaches steady state. *Initial solution:* use Welch's method to identify and discard the transient period.

1.7. Preliminary Work Plan

Both team members will participate in code reviews, debugging sessions, and scheduled progress meetings throughout the project. Version control will be managed through Git to ensure collaborative development and project's traceability for both team members. Also, without damaging work balance some of the specific part of the projects might be given to one team member for example environment setup and sensor application parts can be shared between two members.

REFERENCES

- [1] A. Varga, "OMNeT++ Discrete Event Simulation System," *OpenSim Ltd.*, 2024. [Online]. Available: <https://omnetpp.org/>
- [2] INET Framework for OMNeT++, *OpenSim Ltd.*, 2024. [Online]. Available: <https://inet.omnetpp.org/>
- [3] M. Slabicki, G. Premsankar, and M. Di Francesco, "Adaptive configuration of LoRa networks for dense IoT deployments," in *Proc. IEEE/IFIP NOMS*, 2018, pp. 1–9.
- [4] LoRa Alliance, "LoRaWAN Specification v1.0.3," 2018. [Online]. Available: <https://lora-alliance.org/>
- [5] R. Y. Rubinstein and D. P. Kroese, *Simulation and the Monte Carlo Method*, 3rd ed. Hoboken, NJ: Wiley, 2017.